Lecture 1: Specialization and the Hardware/Software Stack

ECE-GY 9433: Agile System-on-Chip Design

Fall 2025

Prof. Austin Rovinski

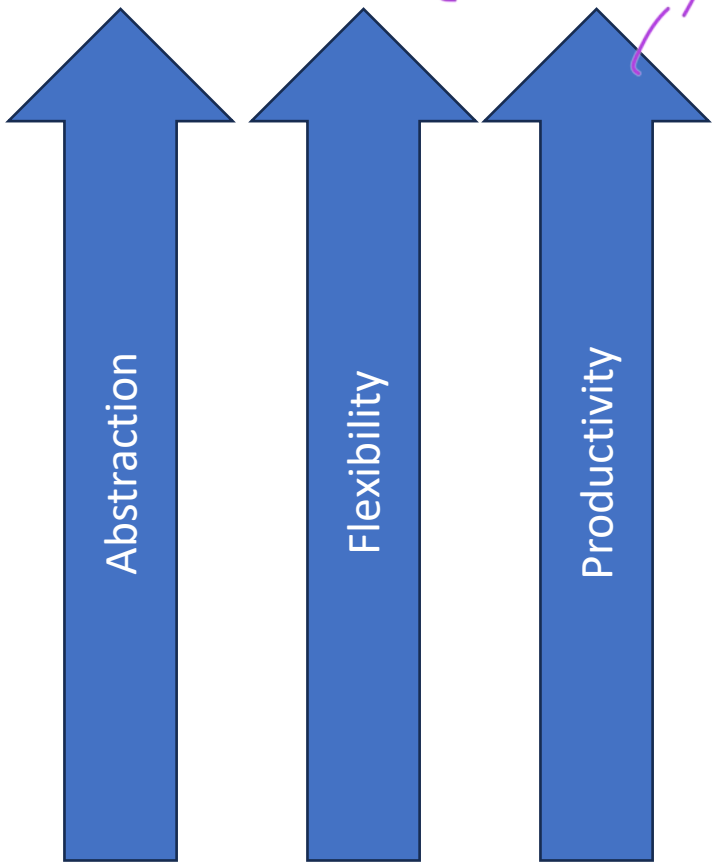
New York University

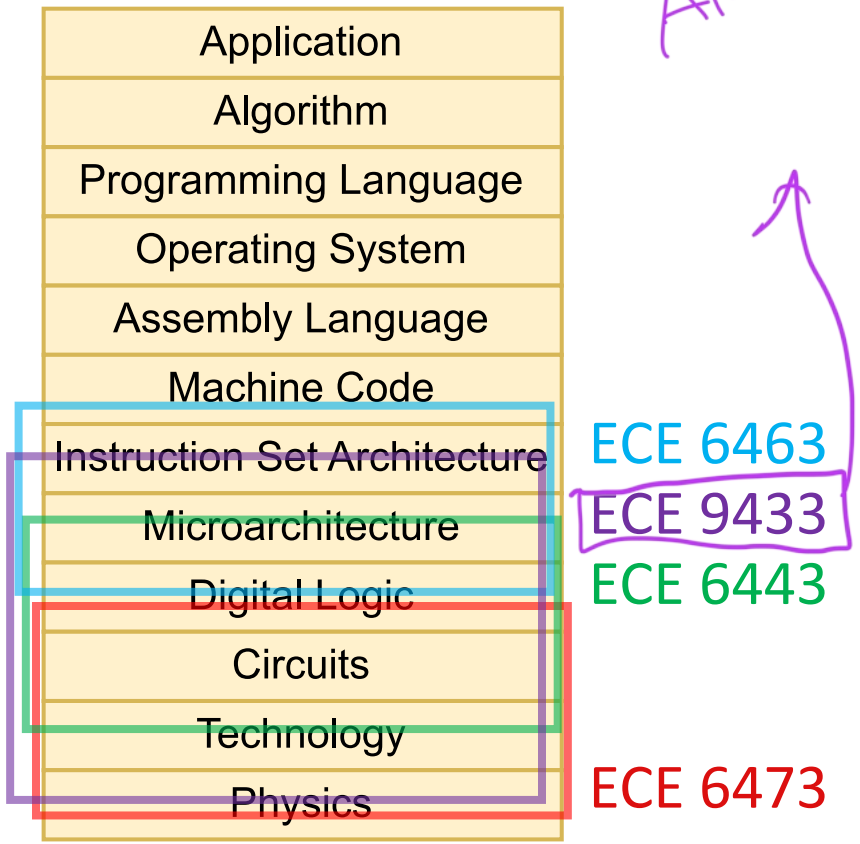
Hardware/Software Stack

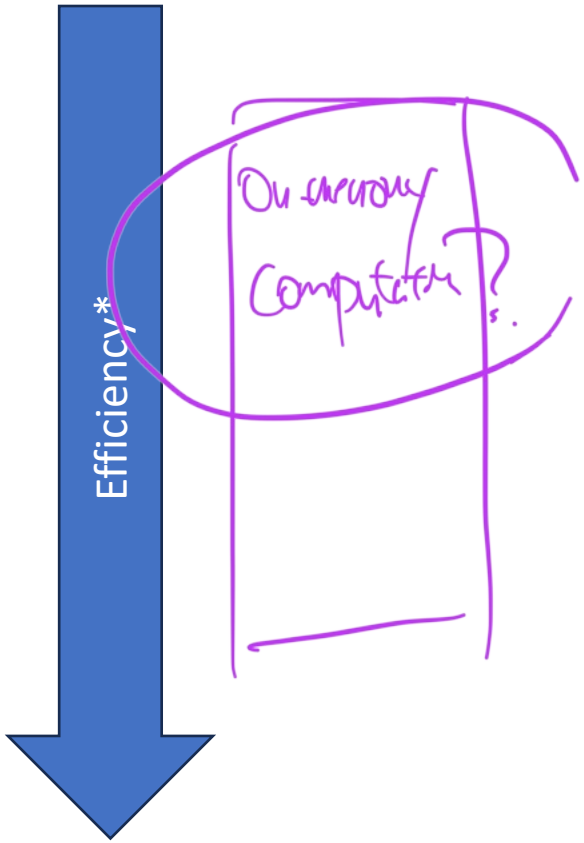
difficulty to change design.

Cost of design.

Almost All





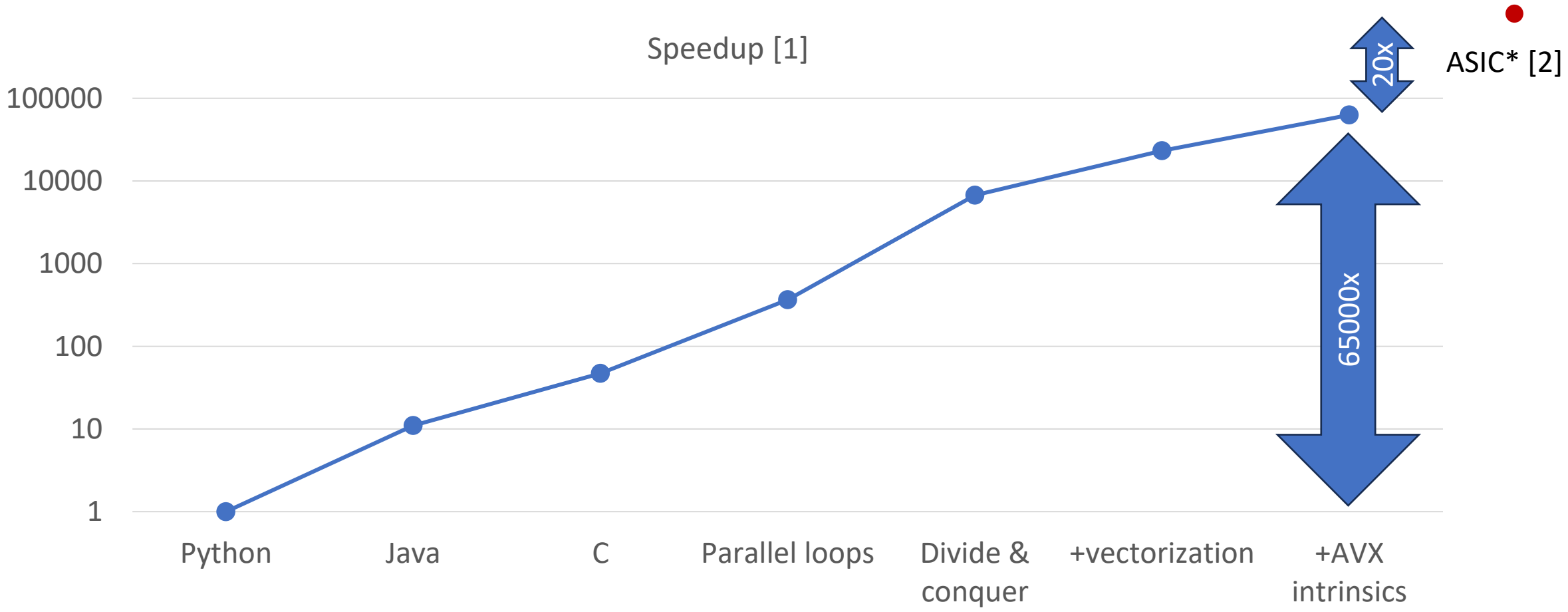


*More efficient when implemented at this level

Changes to algorithm can result in more efficiency gains

Efficiency example:

Quick example: Matrix Multiplication



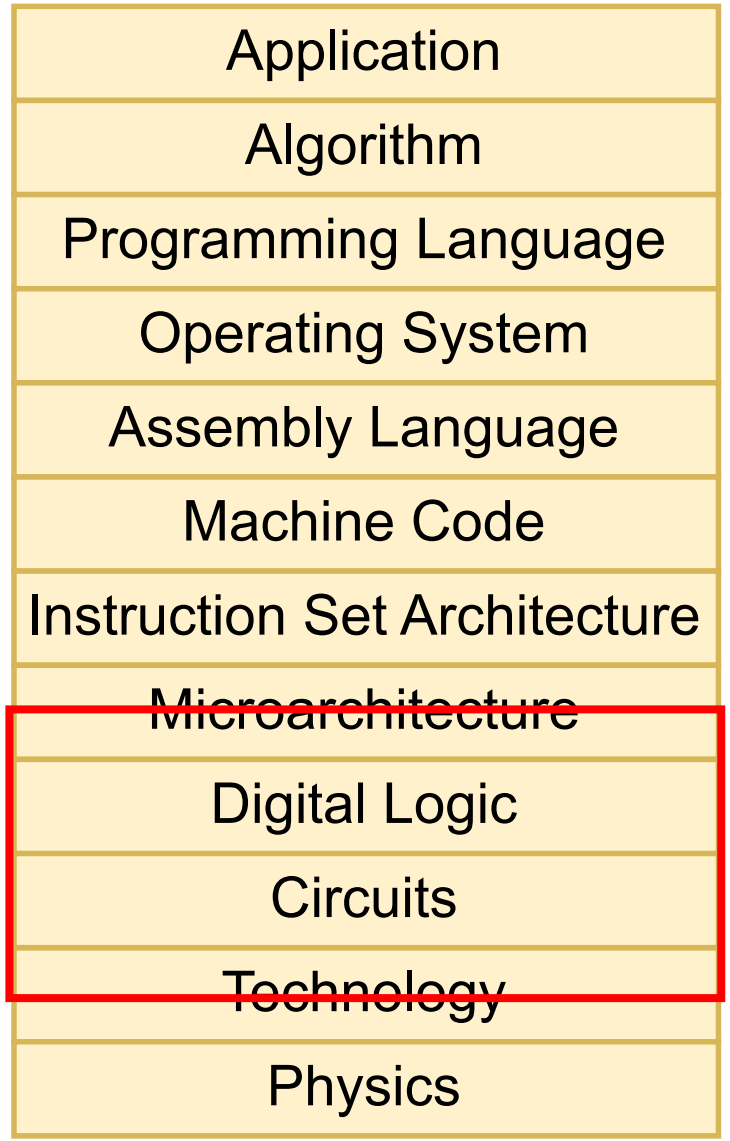
[1] Leierson, et al. Science 2022. [2] Jouppi, et al. ISCA 2017.

*Not normalized for die area, power, cost, etc.

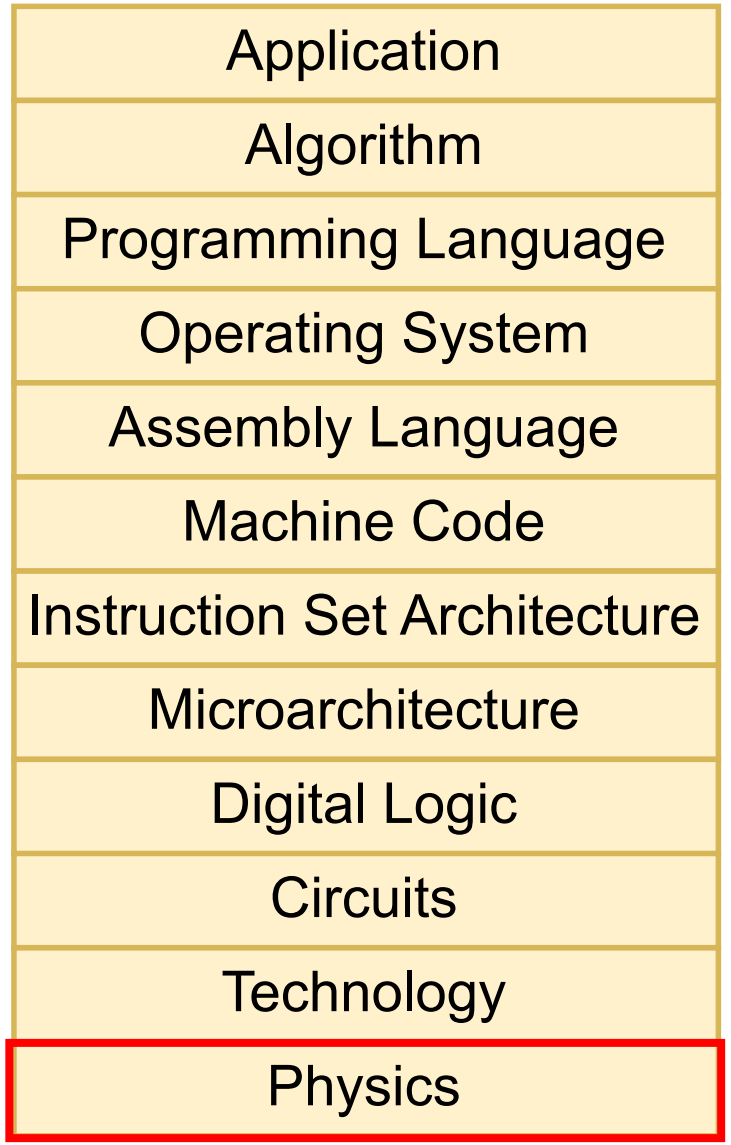
Hardware/Software Stack Review

This class

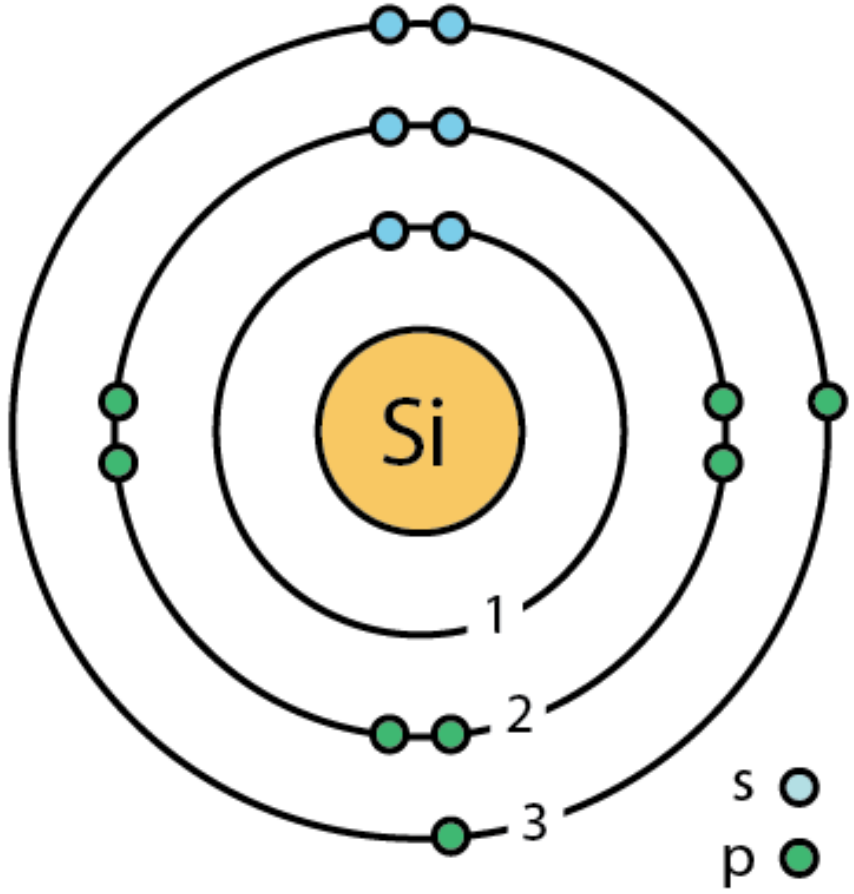
- Conceptual focus on whole system
- Technical focus on “RTL to GDS” – synthesizable code to chip layout files
- Practical focus on lifecycle from idea conception through post-silicon validation

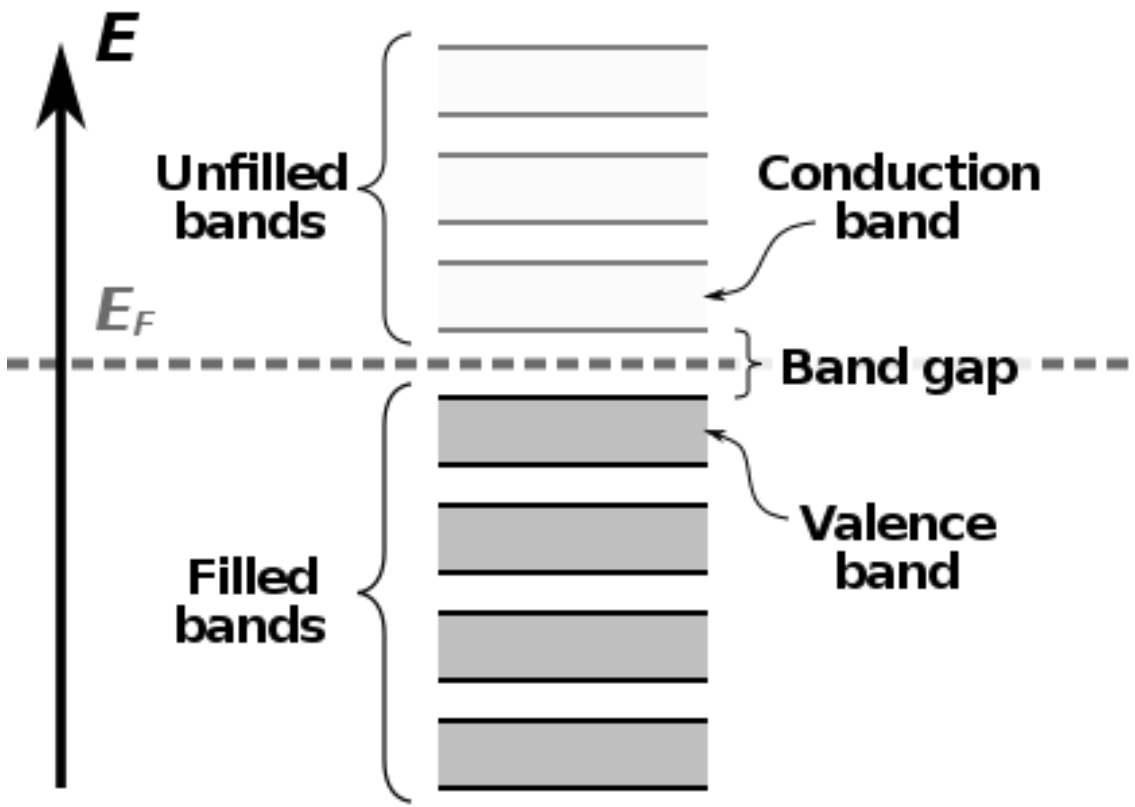


Semiconductors



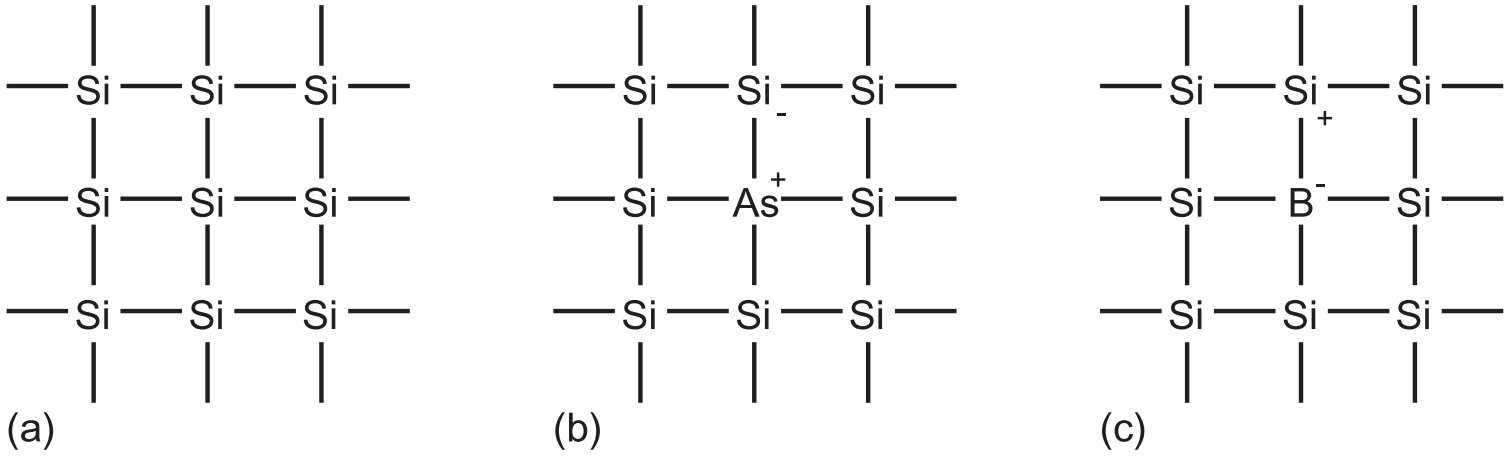
Valence and Conduction Bands

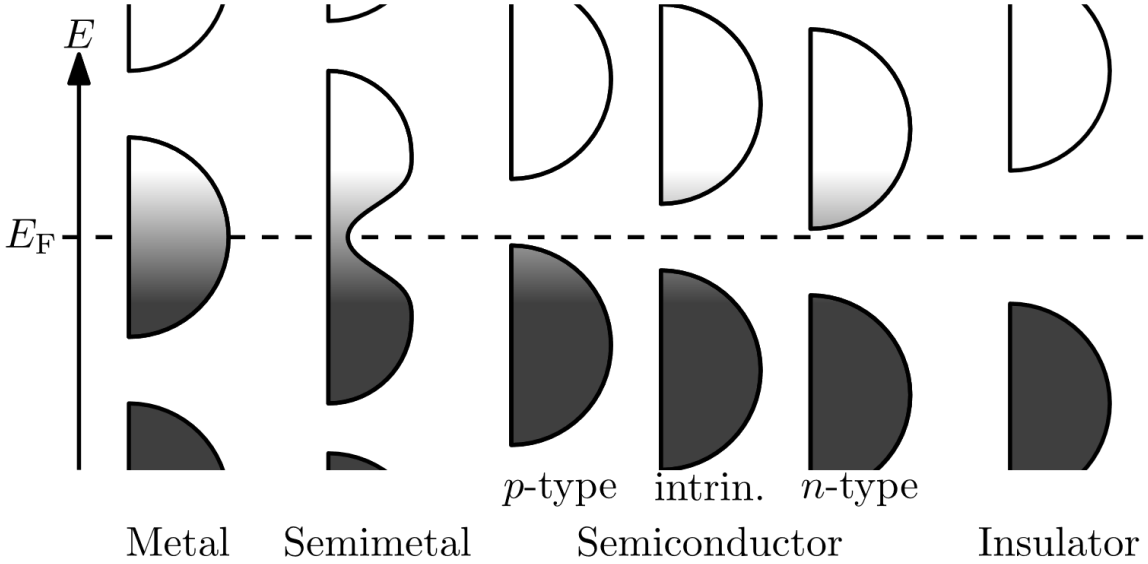




Source: Wikipedia

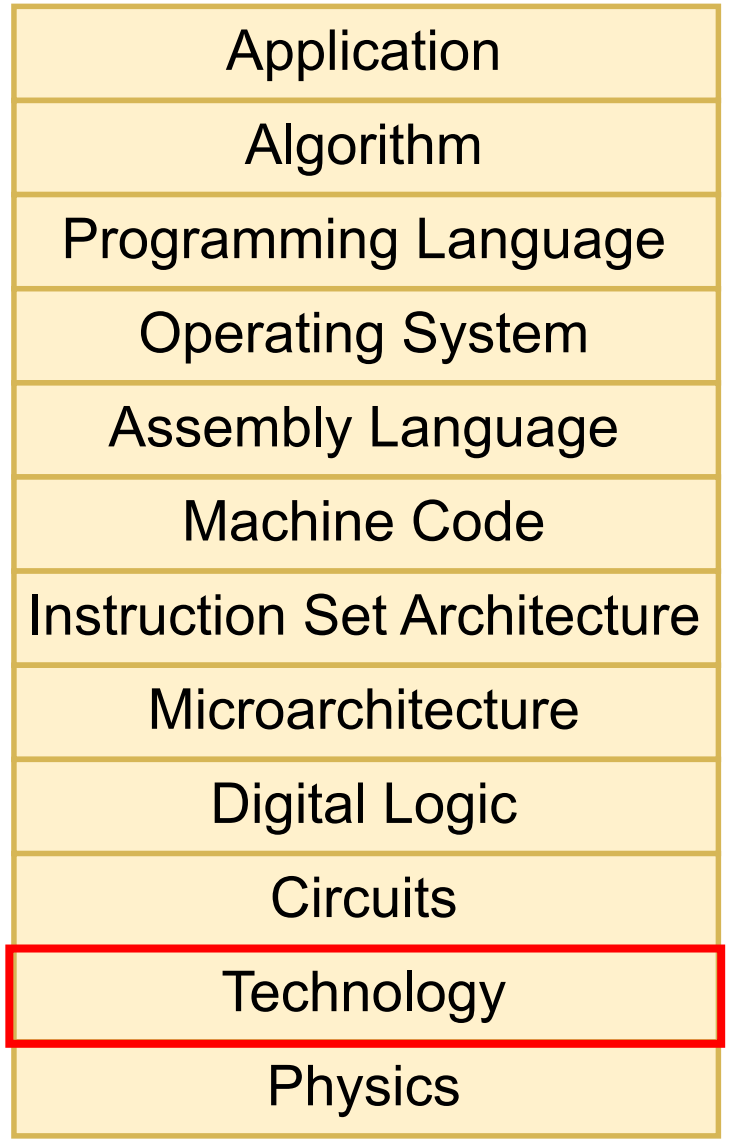
Silicon Crystal Structure



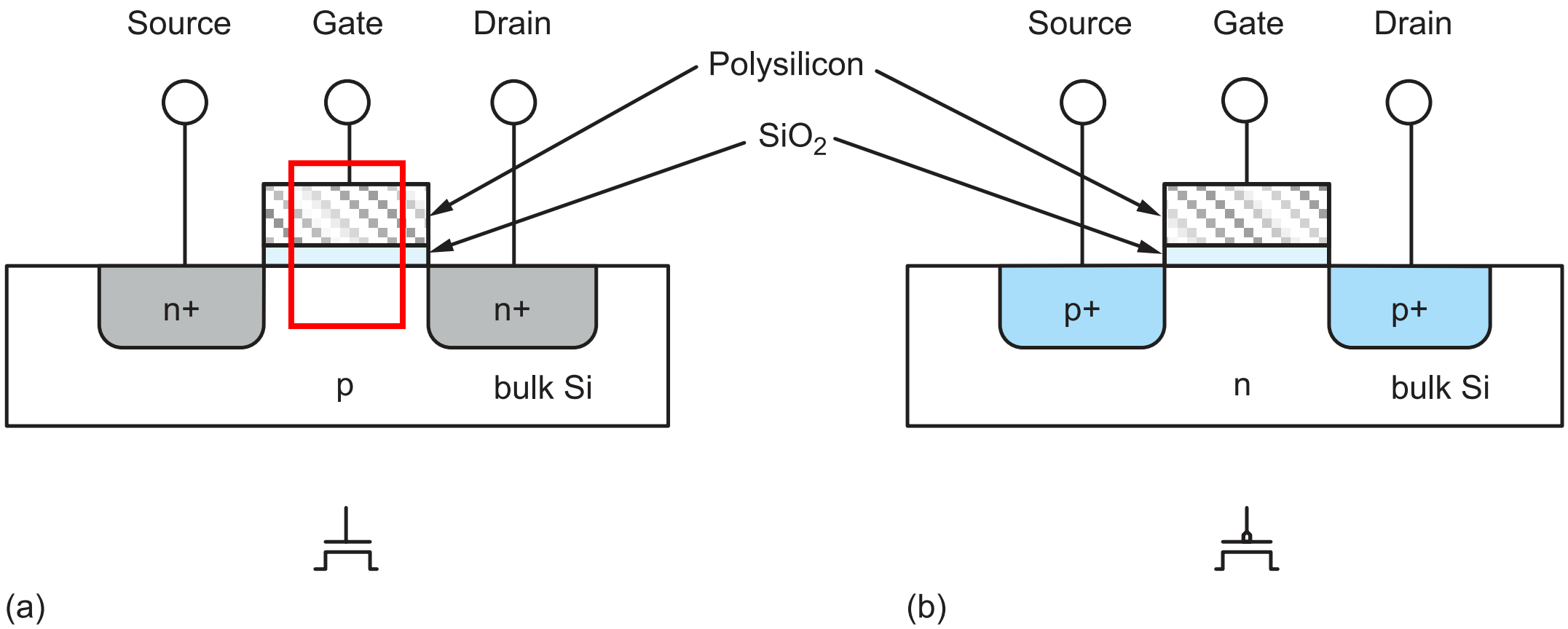


Source: Weste & Harris; Wikipedia

MOSFET Devices



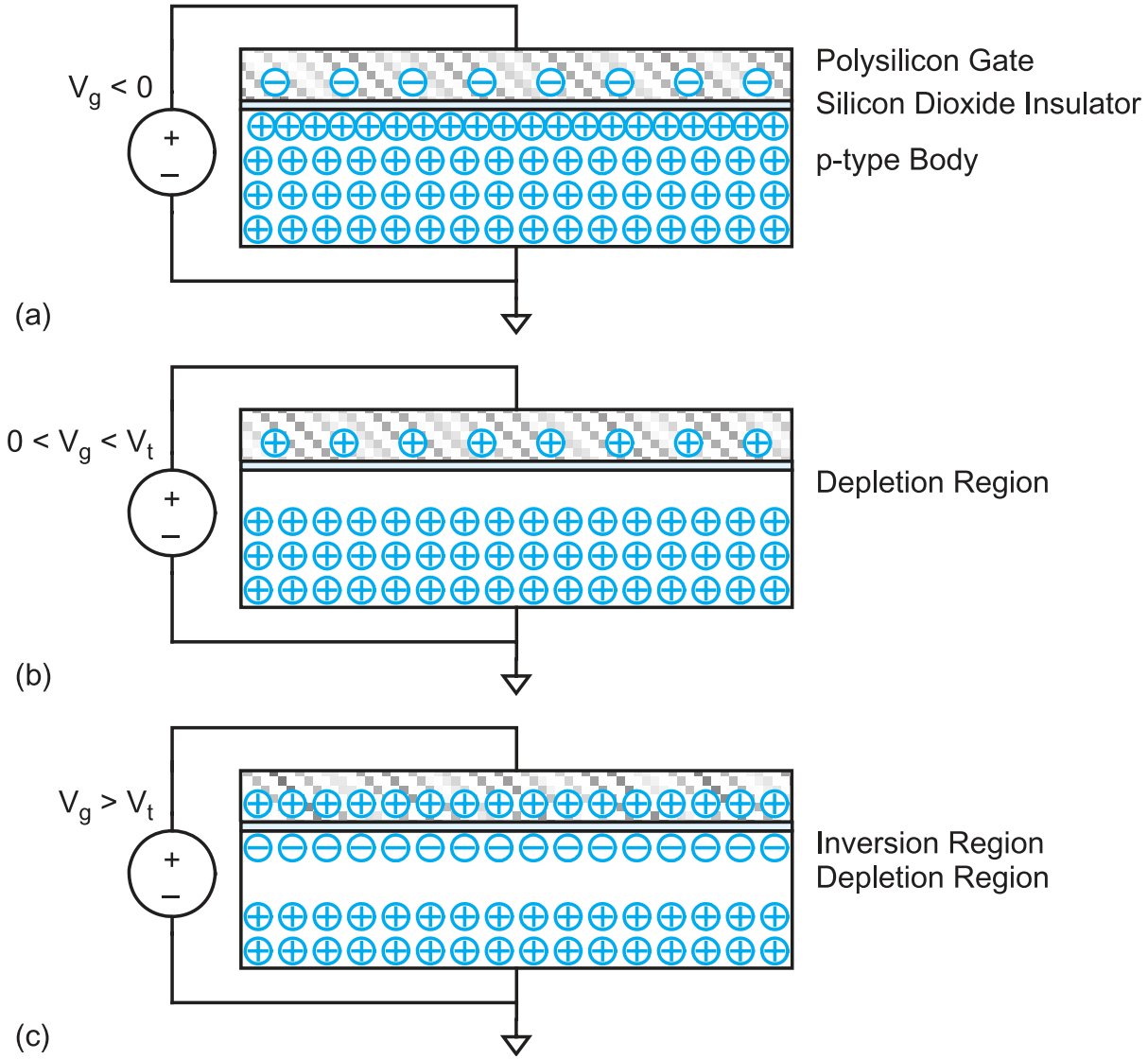
nMOS and pMOS Transistors



(a) (b)

Source: Weste & Harris

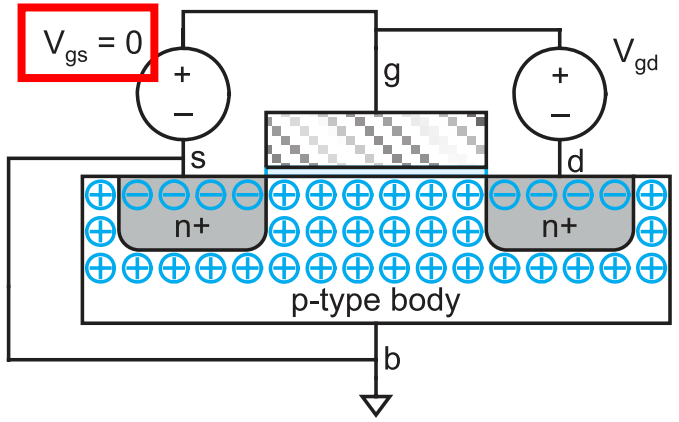
nMOS Device Operating Modes



- Accumulation
  - - charge on gate
  - + charge in channel
  - No conduction
    - n+ source/drain prevent + charge flow
- Depletion
  - + charge on gate
  - Channel empty
    - Dopants balance gate charge
  - No conduction
- Inversion
  - + charge on gate
  - - charge in channel
  - Conduction through free charge carriers

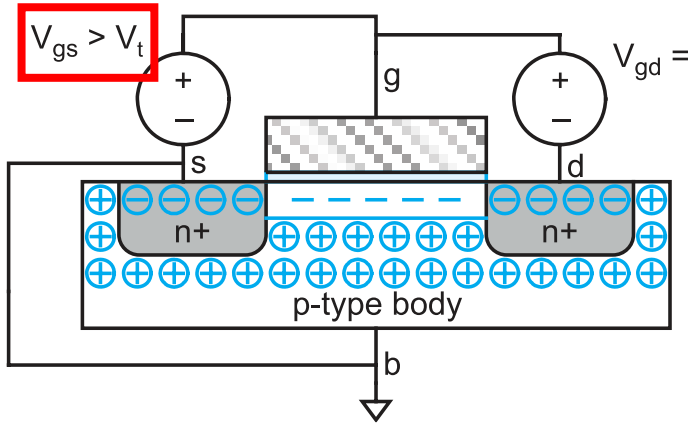
Source: Weste & Harris

nMOS Transistor Operating Regions



(a)

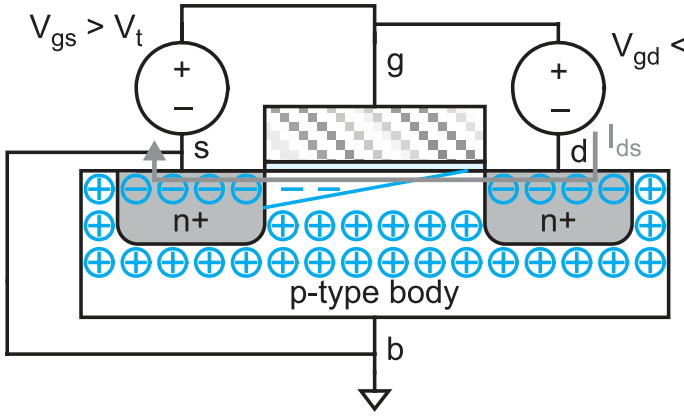
Cutoff:
No Channel
$I_{ds} = 0$



(b)

$V_{ds} = 0$

Linear:
Channel Formed
I_{ds} Increases with V_{ds}

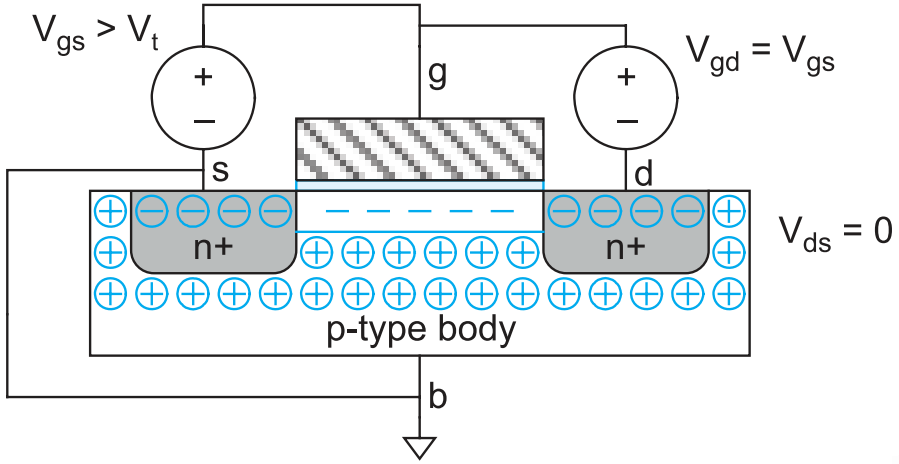


(d)

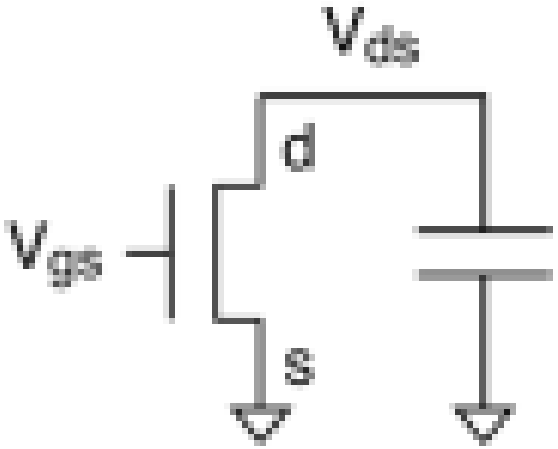
$V_{ds} > V_{gs} - V_t$

Saturation:
Channel Pinched Off
I_{ds} Independent of V_{ds}

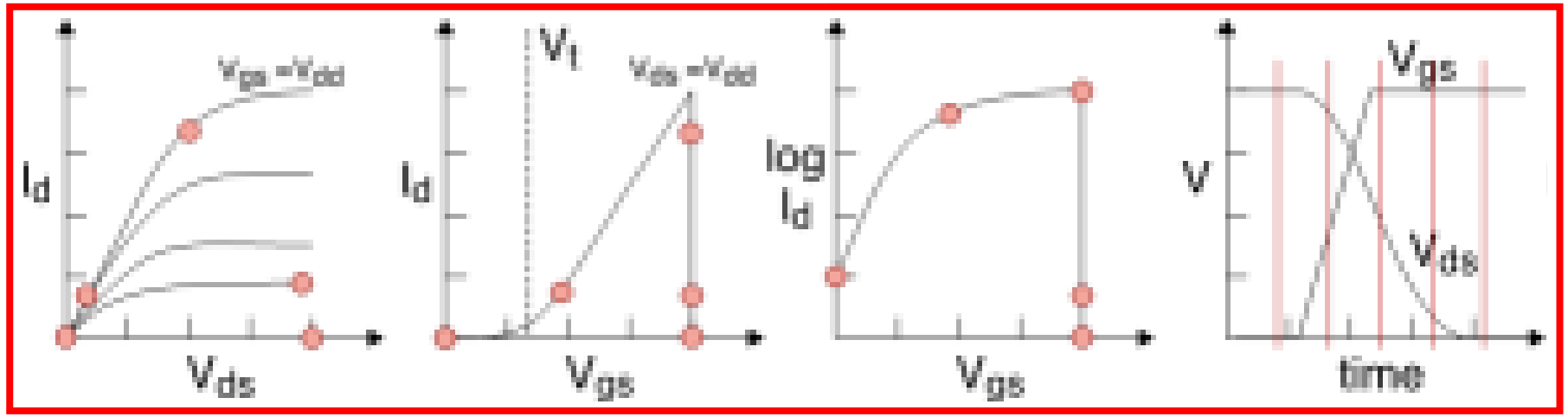
nMOS Transistor Current Behavior



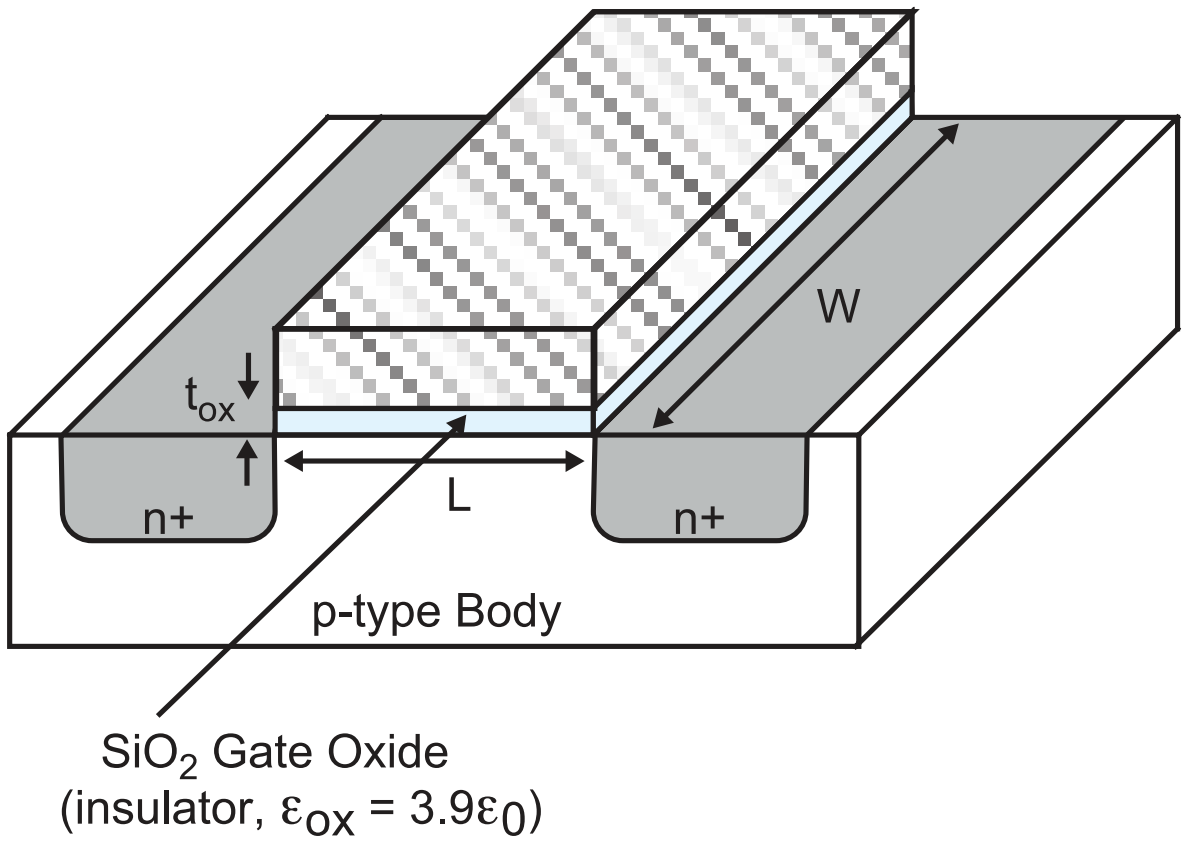
Linear: Channel Formed
I_{ds} increases with V_{ds}



Important takeaway!



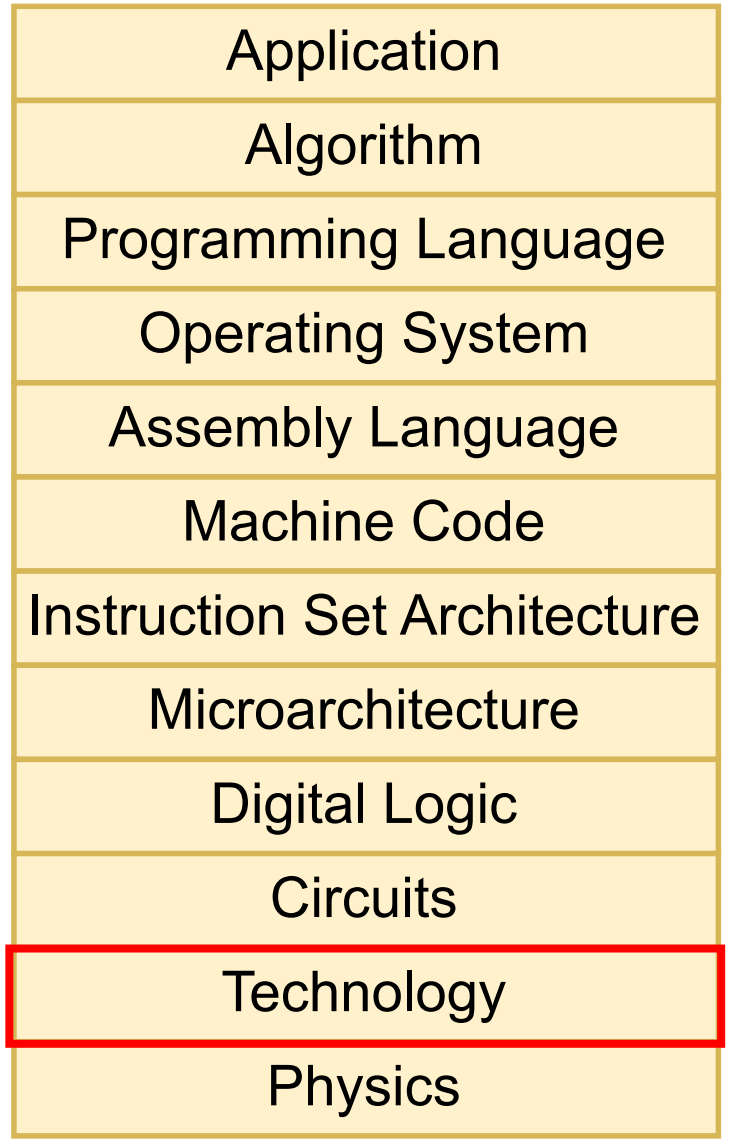
Planar Transistor Scaling



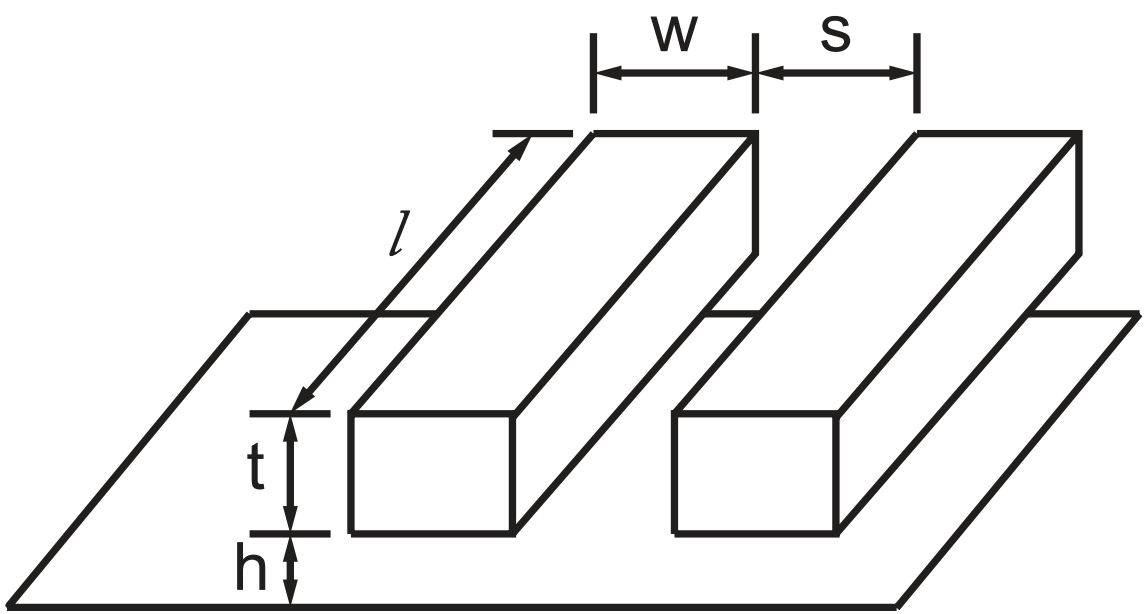
- W: width, L: length,
  C_g: gate capacitance,
  C_d: diffusion capacitance
- $\uparrow W = \uparrow I_d, \uparrow C_d, \uparrow C_g$
  - Faster, larger, more power
- $\uparrow L = \downarrow I_d, \uparrow C_g$
  - Slower, larger, less power

Important takeaway!

Interconnects

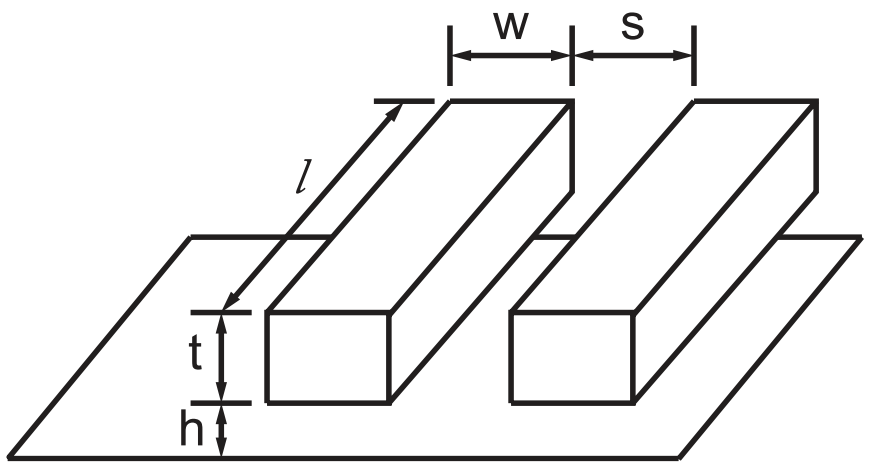


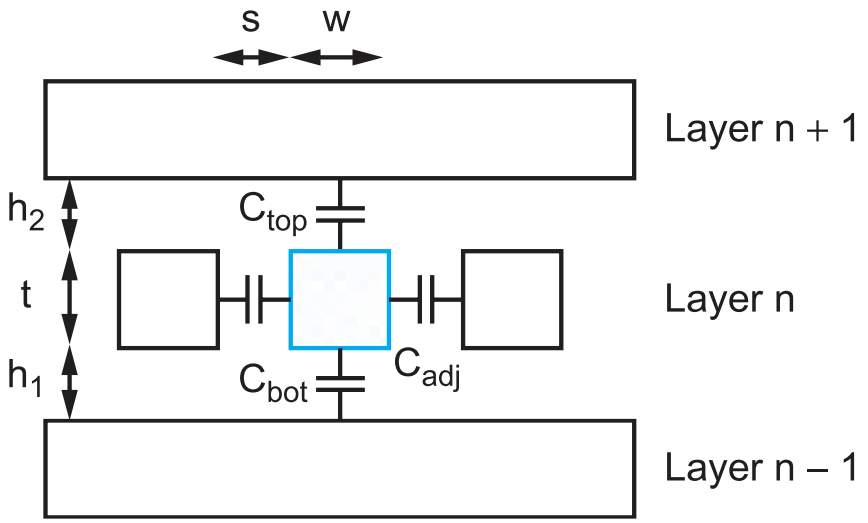
Wire Geometry



- Wires modeled using width W, length l, thickness t
- Horizontal space to other wire s
- Vertical height to other wire h

Wire Parasitics



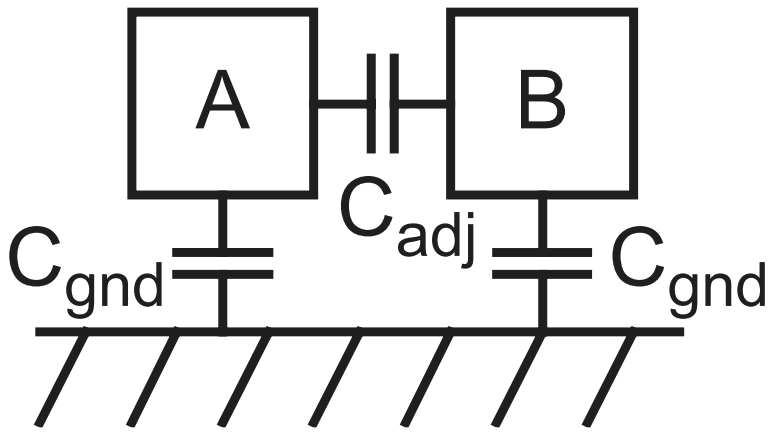


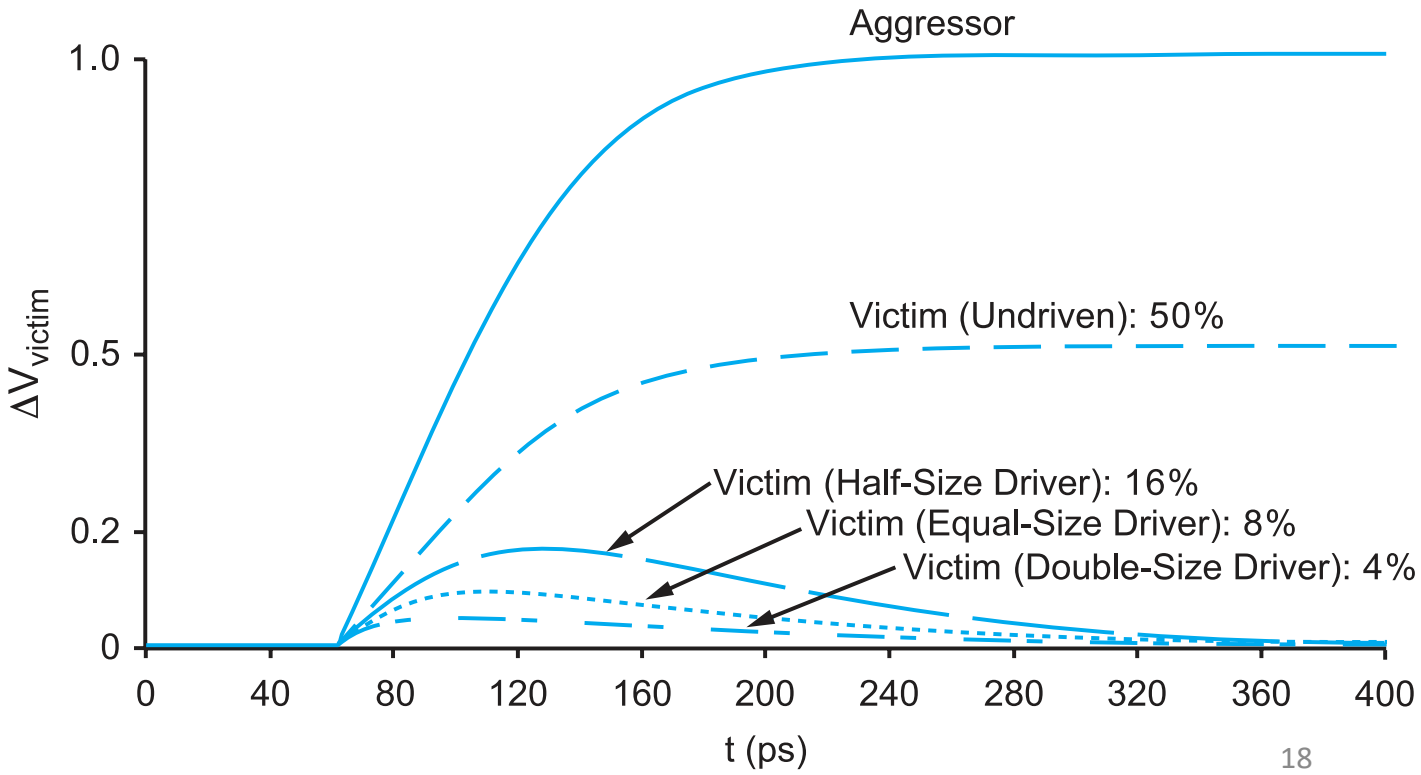
- t_d: wire delay,
  R: wire resistance,
  C: wire capacitance,
- $t_d = RC$
- $\uparrow L = \uparrow R, \uparrow C$
- $\uparrow W = \downarrow R, \uparrow C$
- $\uparrow t = \downarrow R, \uparrow C$

Important takeaway!

Crosstalk

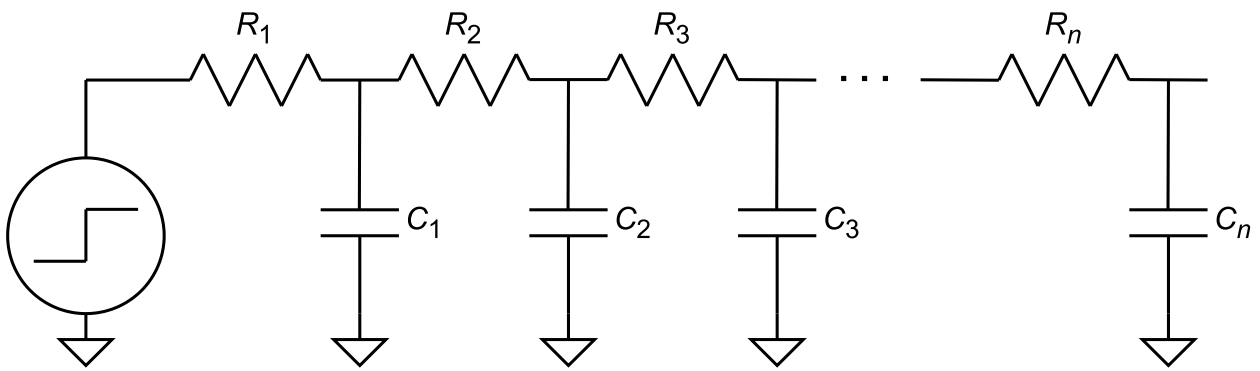
- Pairs of wires have capacitance between them
- Switching wire (aggressor) can cause voltage swings in other wire (victim)
- Variance in delays
- *Glitching* violations
  - Wire changes state

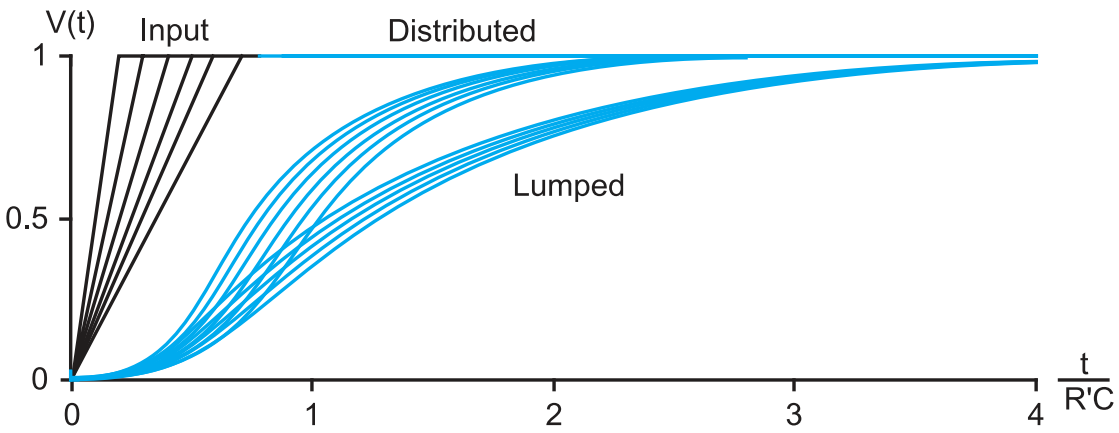




Elmore Delay Model

- Simple (lumped) model: single resistor, single capacitor
- Distributed model more accurate
- $t_d \approx \sum R_i C_i = RC\left(\frac{n+1}{2n}\right) = \frac{RC}{2}$ as $n \to \infty$
- $R, C \propto L$: delay increases *quadratically* with wire length!

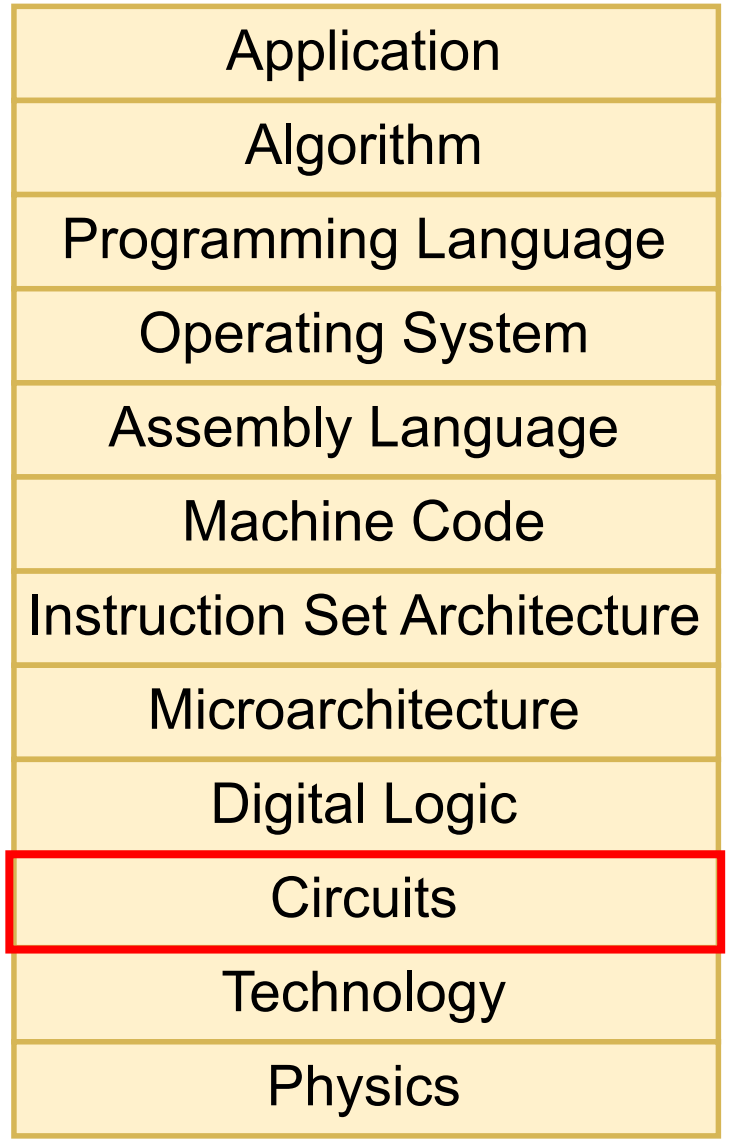




Inductance

- Negligible for on-chip interconnects
- Expensive to compute
- Will come up later for off-chip interconnects

CMOS Circuits



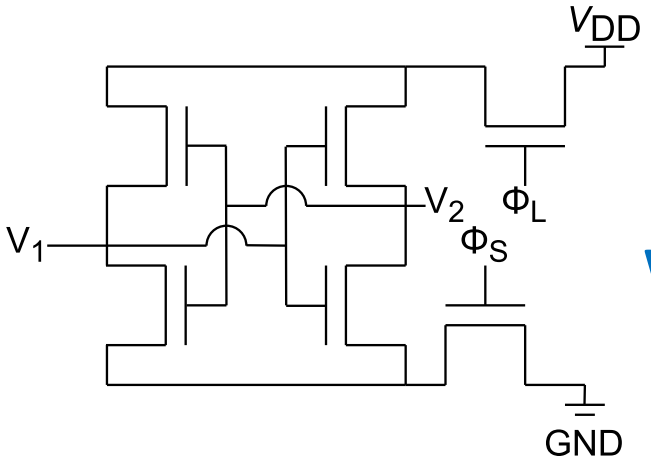
Design Strategies

Full Custom

- Do whatever as long as it follows DRC rules
- Clever designers can put together very efficient circuits
- Costs time and expertise!

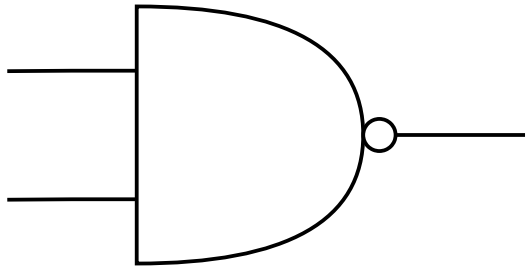
Cell-Based Design

- Make concessions to quantize and abstract design
- Creates rules and constraints for automated algorithms
- Fast and scalable!



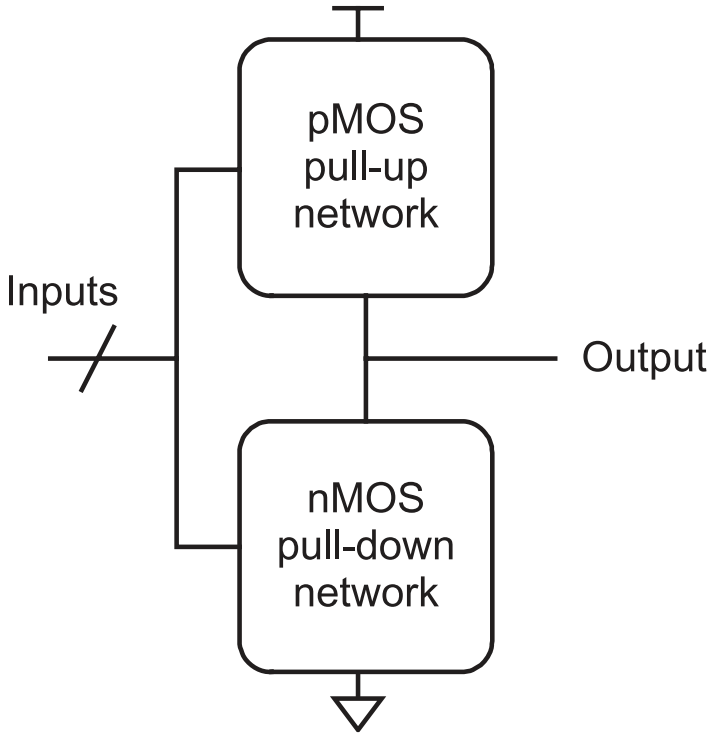
Which one is preferred?

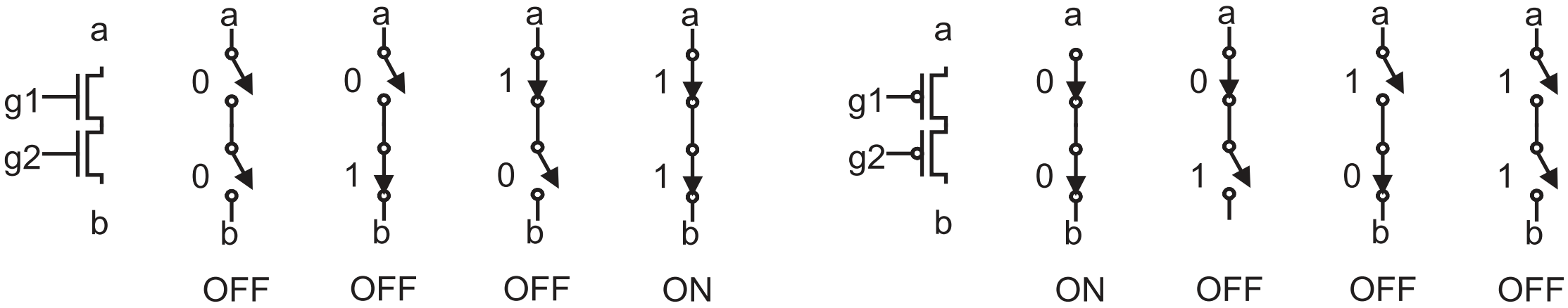
It depends - stay tuned.



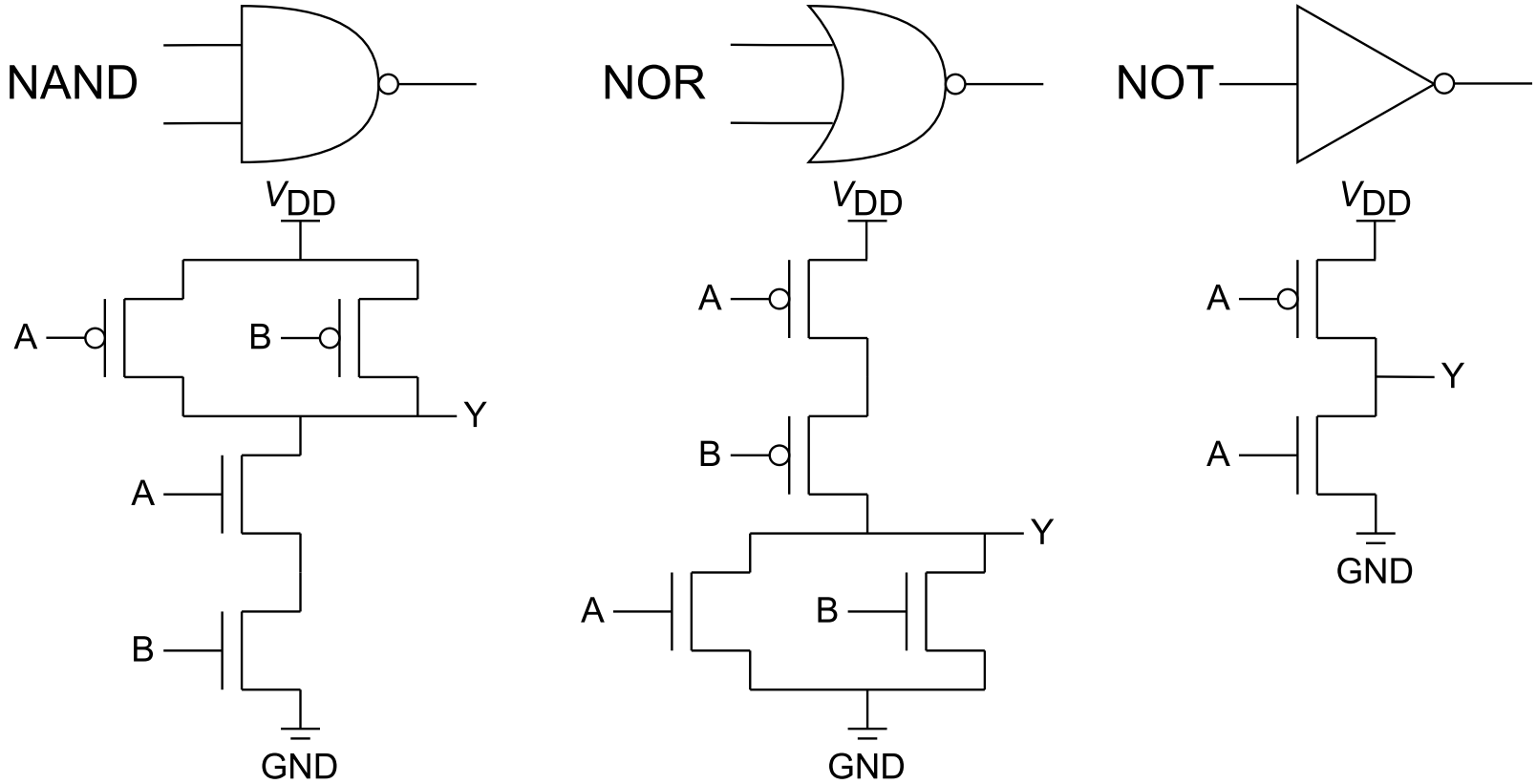
Pull-Up / Pull-Down Networks

- Transistors like switches (on/off)
  - On/off (conducting/nonconducting)
- pMOS conducts from V_{t} to V_{DD}
- nMOS conducts from 0 V to $V_{DD} - V_t$
- For full swing 0 V to V_{DD}
  - pMOS "pulls up" to V_{DD} (logical 1)
  - nMOS "pulls down" to 0 V (logical 0)





Logic Gates



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

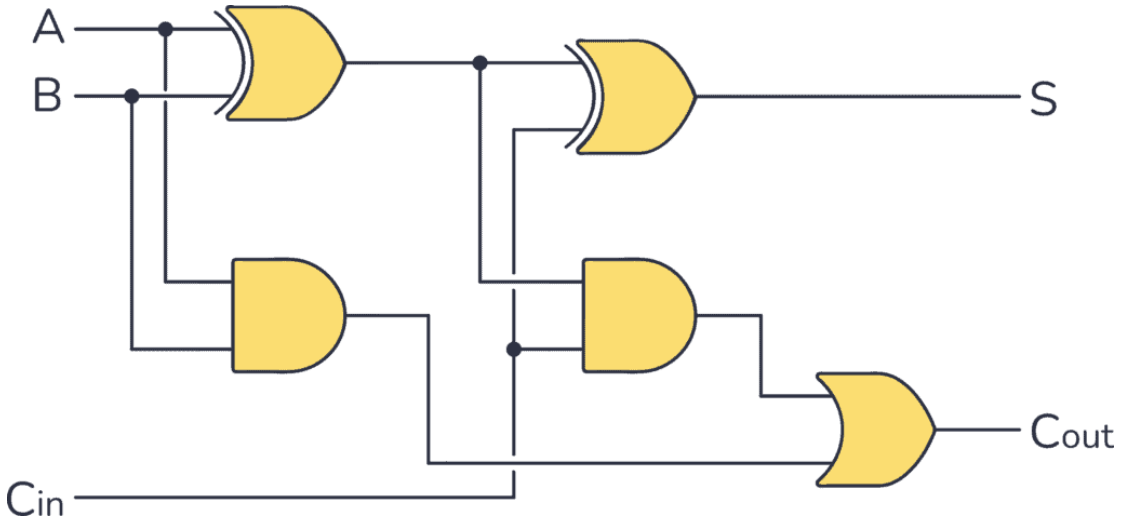
A	Y
0	1

Digital Logic

Application
Algorithm
Programming Language
Operating System
Assembly Language
Machine Code
Instruction Set Architecture
Microarchitecture
Digital Logic
Circuits
Technology
Physics

Combinational Logic

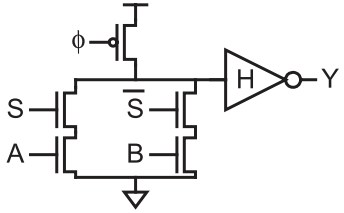
- Chain gates to create complex functions
- Library of cells must be *functionally complete*
  - Definition: Can generate any truth table
  - Must perform inversion
  - Must perform reduction (non-linear)
  - E.g. {AND, NOT}, {NAND}, {NOR}, etc.



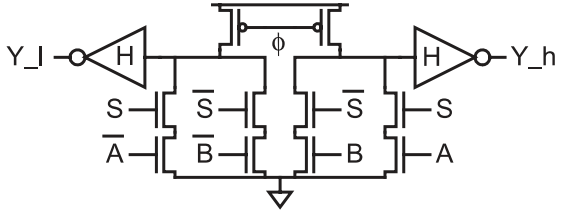
Combinational Logic

- Dozens of other logic families exist
- Very infrequently used
- CMOS has best area/power/noise tradeoff

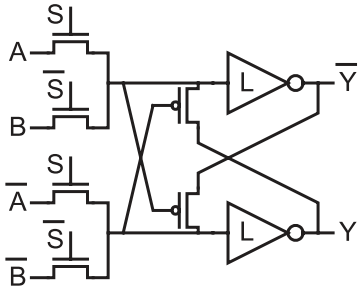
Domino



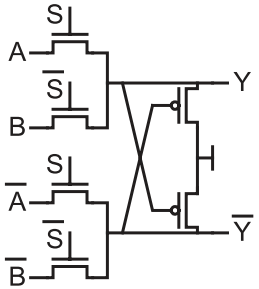
Dual-Rail Domino



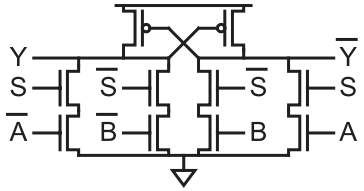
EEPL



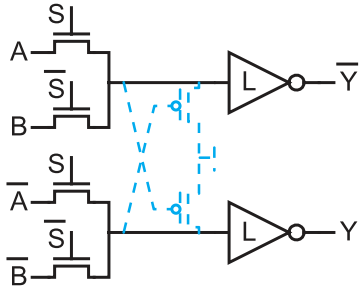
DCVSPG



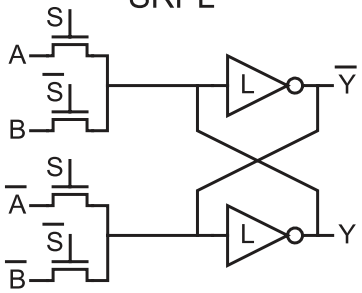
CVSL



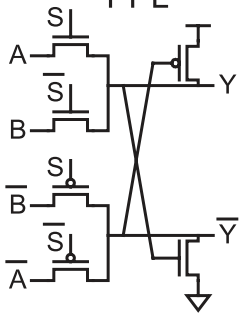
CPL



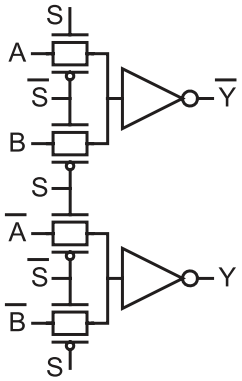
SRPL



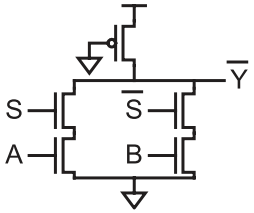
PPL



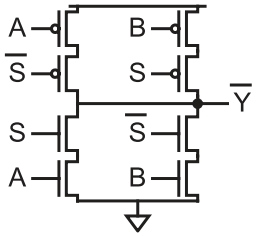
DPL



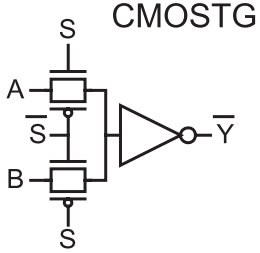
Pseudo-nMOS



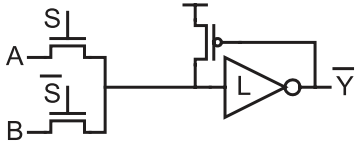
Static CMOS



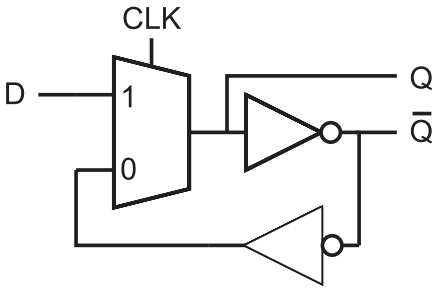
CMOSTG

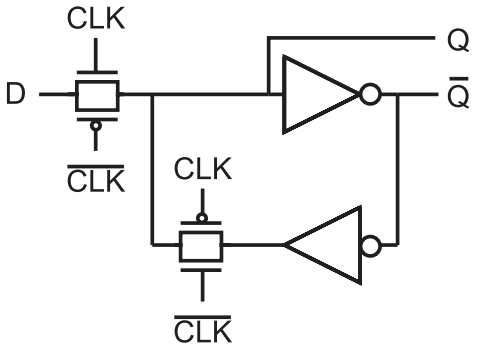


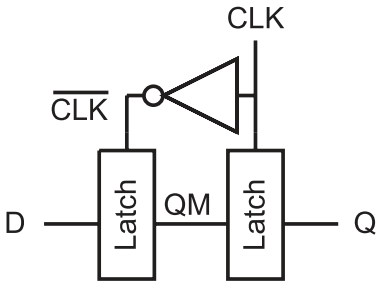
LEAP

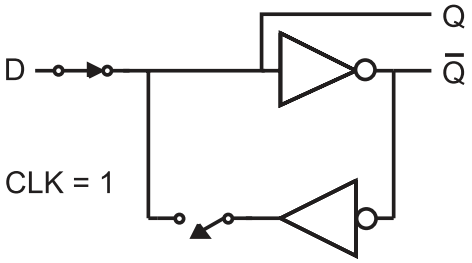


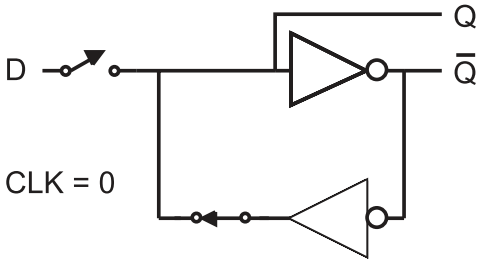
Sequential Logic

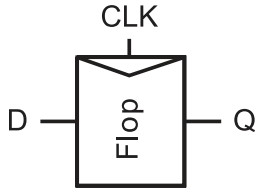


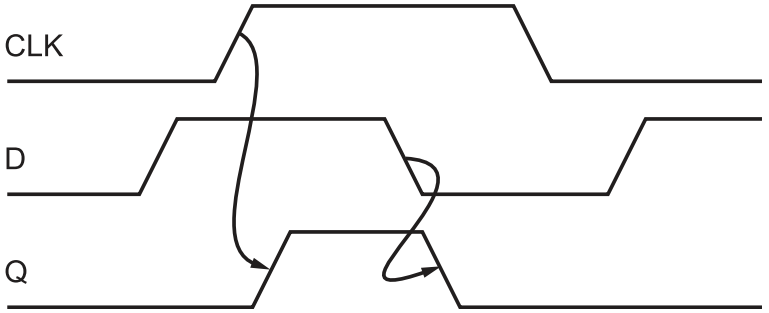


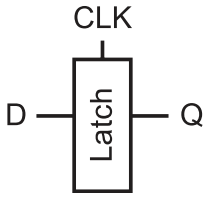


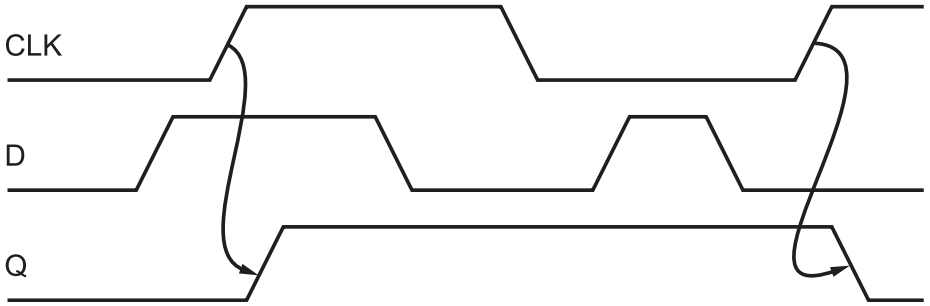






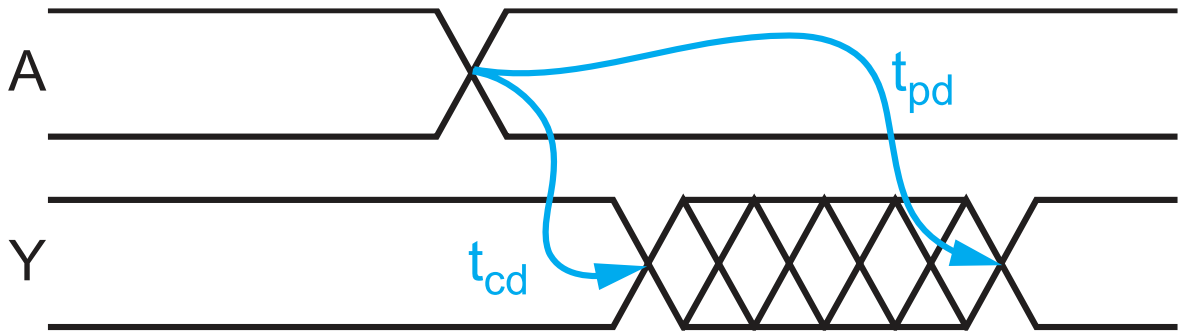






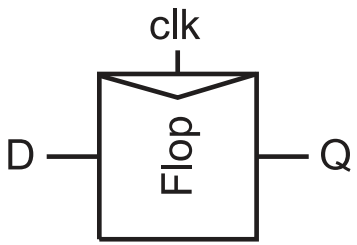
Logic Timing

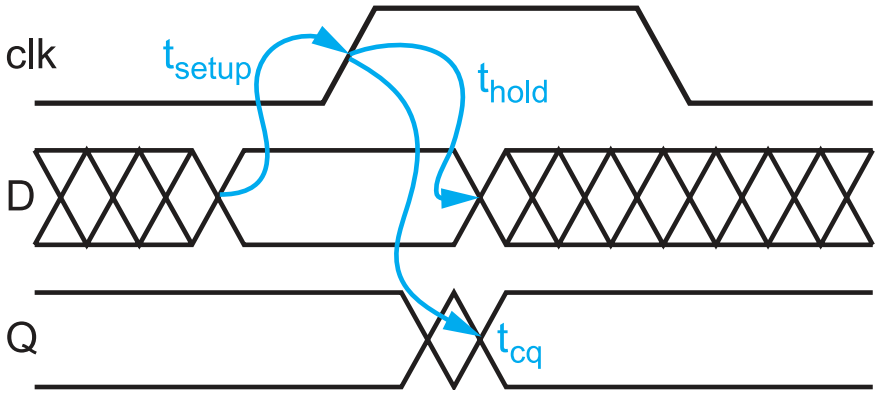
- t_{cd}: **contamination delay**
  - Minimum time that logic can switch
- t_{pd}: **propagation delay**
  - Maximum time that logic can switch



D Flip-Flop Timing

- t_{setup}: **setup time**
  - Minimum time before clock edge which data must arrive
- t_{hold}: **hold time**
  - Minimum time after clock edge arrives which data must remain steady
- t_{cq}: **clock-to-q time**
  - Time after clock arrives where DFF output (Q) changes

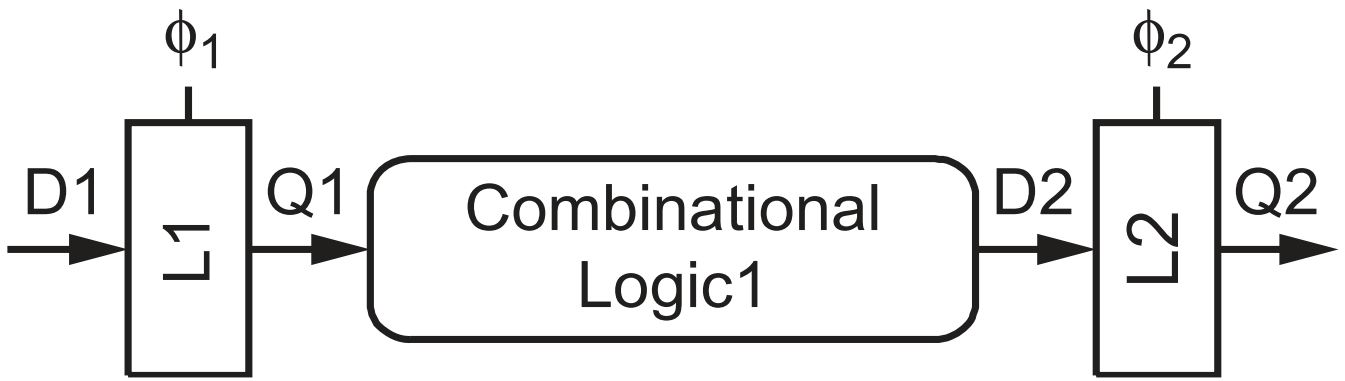




Timing Constraints

- Clock period: T_{c}
- **Slack**: how much you are meeting (or missing) your constraints by
- Setup slack: $T_c - t_{\mathrm{cq}} - t_{\mathrm{pd}} - t_{\mathrm{setup}}$
- Hold slack: $t_{\mathrm{cq}} + t_{\mathrm{cd}} - t_{\mathrm{hold}}$
- Positive slack is good, negative slack is bad
- Ex: $T_c = 500\mathrm{ps}, t_{\mathrm{cq}} = 20\mathrm{ps}, t_{\mathrm{pd}} = 400\mathrm{ps}, t_{\mathrm{setup}} = 2\mathrm{ps}$
  - Setup slack $= 500\mathrm{ps} - 20\mathrm{ps} - 400\mathrm{ps} - 2\mathrm{ps} = 78\mathrm{ps}$
- Ex: $t_{\mathrm{cq}} = 20\mathrm{ps}, t_{\mathrm{cd}} = 20\mathrm{ps}, t_{\mathrm{hold}} = -5\mathrm{ps}$
  - Hold slack $= 20\mathrm{ps} + 20\mathrm{ps} - (-5\mathrm{ps}) = 45\mathrm{ps}$

Clock Skew



- Clocks are not ideal – they have delay like other signals
- **Clock skew**: difference in arrival times between two endpoints (DFFs)
  - Skew $\phi = \phi_2 - \phi_1$
  - Positive ϕ: L1 (launch register) receives clock first
  - Negative ϕ: L2 (capture register) receives clock first
- Setup slack: $T_c - t_{\mathrm{cq}} - t_{\mathrm{pd}} - t_{\mathrm{setup}} + \phi$
  - Positive skew helps, negative skew hurts
- Hold slack: $t_{\mathrm{cq}} + t_{\mathrm{cd}} - t_{\mathrm{hold}} - \phi$
  - Negative skew helps, positive skew hurts

Important takeaway!

Microarchitecture

Application
Algorithm
Programming Language
Operating System
Assembly Language
Machine Code
Instruction Set Architecture
Microarchitecture
Digital Logic
Circuits
Technology
Physics

Microarchitectural Strategies for Efficiency

- Caching
  - Move things you need closer to where you need them
- Parallelizing
  - Break operations into multiple pieces and execute at the same time
- Speculating
  - Predict work that needs to be done in the future
- Specializing
  - Eliminate operations / structures that aren’t needed

References

- C. Batten, Class Lecture, Topic: “CMOS Devices.” ECE 5745, School of Electrical and Computer Engineering, Cornell University, Ithaca, NY, 2023.
- N. Weste and D. Harris, “CMOS VLSI Design: A Circuits and Systems Perspective,” 4th Edition, Pearson Addison Wesley, Boston, 2010.